Safe Project Architecture

Project documentation companion

April 21, 2026

Contents

1 Project Application **2**

2 Hardware Platform **2**

2.1 Reference Hardware 2

3 Hardware Architecture **4**

3.1 System View 4

3.2 Control Partitioning 4

3.3 Hardware Blocks and Responsibilities 4

3.4 I/O Mapping 5

3.5 Electrical Interaction Model 6

4 Firmware Architecture **6**

4.1 Layered View 6

4.2 Main Execution Model 7

4.3 Firmware Data Flow 7

4.4 Core Modules 7

4.5 Application States 9

4.6 User and Admin Interaction Model 9

5 Security-Relevant Behavior **10**

6 Relevant Source Map **10**

7 Summary **11**

1 Project Application

This project implements the firmware for a prototype electronic safe controller built on an STM32 NUCLEO-L476RG platform. The application combines authentication, lockout protection, operator feedback, and a small administrator console in a single embedded control loop.

The current prototype demonstrates these capabilities:

- user PIN entry through a 4x4 matrix keypad
- operator and user feedback on a 4-digit seven-segment display
- LED and buzzer indication for armed, admin, success, error, and lockout conditions
- configurable timed lockout after repeated failed attempts
- administrative functions for PIN change, lockout configuration, sound control, and diagnostics

At the application level, the system behaves like the front panel of an electronic safe:

- normal users enter a 4-digit PIN
- administrators enter a 6-digit PIN to reach a maintenance menu
- the display and indicators continuously reflect the current security state
- the firmware enforces a lockout after too many failed attempts

2 Hardware Platform

The hardware is organized around three visible elements:

- an STM32 NUCLEO-L476RG development board as the MCU and timing/control platform
- an Arduino-style Multi-Function Shield for display, LEDs, buzzer, and three push buttons
- an external 4x4 matrix keypad connected to free Morpho header pins

2.1 Reference Hardware

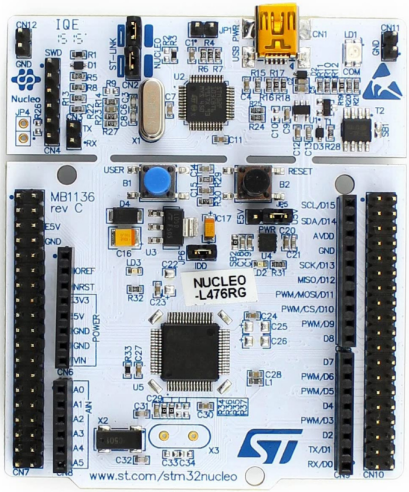

Figure 1: STM32 NUCLEO-L476RG development board used as the MCU host platform.

Figure 2: Multi-Function Shield providing the 4-digit display, LEDs, buzzer, and buttons.

Figure 3: 4x4 matrix keypad used as the primary secure input device.

3 Hardware Architecture

3.1 System View

User / Administrator	Operates the front panel, enters user or admin credentials, and receives status feedback.
Matrix keypad	Primary secure input surface for digits `0-9`, control keys `*` and `#`, and function keys `A-D`.
Shield push buttons	Secondary shortcut inputs mapped by firmware to logical keys `A`, `#`, and `*`.
STM32L476RG MCU	Schedules periodic activities, scans inputs, runs the state machine, and drives all outputs.
Seven-segment display	Presents messages such as `SAFE`, `OPEN`, `FAIL`, timers, and menu labels.
LEDs and buzzer	Provide status indication for armed, admin, success, error, and lockout conditions.

3.2 Control Partitioning

The design separates logical behavior from hardware-specific I/O:

- `src/app_fsm.c` owns safe behavior, security states, and user/admin workflows
- `src/hw.c` owns GPIO configuration, keypad scanning, display multiplexing, LEDs, buzzer, and low-power waiting
- `src/display.c`, `src/indicators.c`, `src/keypad.c`, and `src/storage.c` define logical models that decouple the application from register-level code

This partitioning is important because the safe logic can be validated independently from the board, while the board support layer contains the timing-sensitive electrical behavior.

3.3 Hardware Blocks and Responsibilities

Block	Role in the system	Firmware boundary
STM32L476RG	Main controller, scheduler clock source, GPIO host, and low-power execution target	`src/main.c`, `src/hw.c`
Multi-Function Shield display	4-digit numeric and text feedback	`src/display.c` and display drive logic in `src/hw.c`
Shield LEDs	Armed, admin, success, error, and lockout indication	`src/indicators.c` and LED mapping in `src/hw.c`

Block	Role in the system	Firmware boundary
Shield buzzer	Audible keypress and alarm feedback	`src/indicators.c` and buzzer pulse logic in `src/hw.c`
Shield buttons	Shortcut inputs mapped to logical keys `A`, `#`, and `*`	debounce and key injection logic in `src/hw.c`
4x4 keypad	Primary front-panel input	matrix scan and debounce logic in `src/hw.c`

3.4 I/O Mapping

The implementation in `src/hw.c` defines the active hardware mapping used by the prototype.

Shield connections

Function	MCU pin	Notes
Shift register latch	`PB5`	Shield display drive
Shift register clock	`PA8`	Shield display drive
Shift register data	`PA9`	Shield display drive
Buzzer	`PB3`	Active-low behavior selected in `include/config.h`
LED1	`PA5`	Error and lockout patterns
LED2	`PA6`	Armed and admin patterns
LED3	`PA7`	Admin patterns
LED4	`PB6`	Success indication
Shield button S1	`PA1`	Injected as key `A`
Shield button S2	`PA4`	Injected as key `#`
Shield button S3	`PB0`	Injected as key `*`

External keypad connections

Keypad signal	MCU pin
Row 1	`PC10`
Row 2	`PC11`
Row 3	`PC12`
Row 4	`PD2`
Column 1	`PC8`
Column 2	`PC6`
Column 3	`PC5`
Column 4	`PC4`

Keypad map

```
1 2 3 A
4 5 6 B
7 8 9 C
* 0 # D
```

3.5 Electrical Interaction Model

The hardware layer performs these low-level tasks:

- configures GPIO for output, pull-up input, or analog high-impedance mode
- scans keypad rows against pulled-up columns and accepts exactly one stable pressed key
- debounces both keypad and shield buttons before generating logical key events
- multiplexes the four display digits at a fixed refresh cadence
- converts ASCII characters into seven-segment patterns
- drives LED patterns according to logical application state
- pulses the buzzer for keypress, success, error, or periodic lockout feedback
- sleeps with `__WFI()` between scheduled activities

One notable implementation detail is the buzzer idle strategy: when the buzzer is silent, its pin is parked in analog mode so the shield sees a high-impedance state instead of a guessed inactive logic level.

4 Firmware Architecture

4.1 Layered View

`src/main.c`	Deterministic superloop scheduler and timing orchestration.
`src/hw.c`	Board support layer: GPIO setup, input scan, display refresh, LED drive, buzzer drive, and sleep behavior.
`src/app_fsm.c`	Application state machine and security policy.
`src/display.c`, `src/indicators.c`, `src/keypad.c`, `src/storage.c`	Logical models and data abstractions consumed by the application and hardware layers.

4.2 Main Execution Model

The firmware is single-threaded and built around a deterministic superloop. `main()` performs three periodic activities using millisecond time derived from SysTick:

Activity	Period	Purpose
Input scan	`1 ms`	Poll keypad matrix and shield buttons
App tick	`50 ms`	Process queued key events and advance the state machine
Display/output refresh	`2 ms`	Multiplex display digits and apply LED/buzzer outputs

This timing model keeps acquisition, policy, and output refresh separate:

- input acquisition stays fast enough for responsive key handling
- application policy remains coarse-grained and easier to reason about
- display multiplexing remains independent from business logic timing

4.3 Firmware Data Flow

1. `hw_poll_inputs()` scans the physical keypad and push buttons.
2. Stable physical events are translated into logical keys and pushed into the keypad queue.
3. `app_tick()` drains the queue and runs the state machine.
4. The state machine updates logical display, indicator, and storage state.
5. `hw_apply_outputs()` reads the logical output intent and drives the display, LEDs, and buzzer.

4.4 Core Modules

`src/main.c`

Provides the scheduler shell:

- initializes the hardware boundary and application model
- tracks last execution time for each periodic task
- runs periodic input polling, application ticking, and output refresh
- enters low-power wait between events

`src/app_fsm.c`

Implements the safe's control policy as a finite-state machine. It is the main source of truth for security behavior and operator interaction.

Main responsibilities:

- accept logical key events

- validate user and admin PINs
- maintain state transitions and timeouts
- control lockout behavior
- manage admin menu actions
- render logical display text and indicator patterns

`src/storage.c`

Stores mutable configuration and counters:

- user PIN
- admin PIN
- lockout duration
- sound enabled flag
- maximum failed attempts
- failed-attempt counter

The current storage model is volatile only. A power cycle restores default values. The module shape intentionally allows a later replacement with non-volatile persistence without changing the application contract.

`src/keypad.c`

Provides a bounded key event queue. This prevents the state machine from depending directly on scan timing and gives the application one stable input abstraction.

`src/display.c`

Maintains a logical four-character display buffer with helpers for fixed text, numeric rendering, and masked progress feedback during PIN entry.

`src/indicators.c`

Maintains the logical LED and buzzer intent chosen by the state machine, including enforcement of the global sound enable flag.

`src/hw.c`

Translates logical display and indicator intent into physical GPIO behavior and converts physical input activity into stable logical key events.

4.5 Application States

State	Meaning
`STATE_IDLE`	Armed and waiting for input
`STATE_ENTER_PIN`	User PIN entry in progress
`STATE_VALIDATE_PIN`	PIN verification step
`STATE_ACCESS_GRANTED`	Transient success state
`STATE_ACCESS_DENIED`	Transient failure state
`STATE_LOCKOUT`	Timed lockout after too many failed attempts
`STATE_ADMIN_AUTH`	Admin PIN entry
`STATE_ADMIN_MENU`	Administrator menu selection
`STATE_CHANGE_PIN`	User PIN update workflow
`STATE_SET_LOCKOUT`	Lockout duration configuration workflow
`STATE_DIAGNOSTICS`	Failure-count and status summary view

4.6 User and Admin Interaction Model

User path

1. The display shows `SAFE`.
2. The user enters four digits.
3. `#` submits the PIN.
4. A valid PIN shows `OPEN` and success feedback.
5. An invalid PIN shows `FAIL`.
6. After the configured attempt threshold, the safe enters timed lockout and the display shows the remaining seconds.

Admin path

1. The operator enters admin mode with `A`.
2. A 6-digit admin PIN is entered and submitted with `#`.
3. On success, the firmware enters `STATE_ADMIN_MENU`.
4. `B` and `C` cycle forward and backward through menu items.
5. `#` activates the selected function.
6. `*` returns to idle or backs out of the current admin workflow.

Available admin menu functions:

- `PINC`: change the user PIN
- `TIME`: set lockout duration in seconds

- `BEEP`: enable or disable sound feedback
- `FAIL`: show failed-attempt count
- `DIAG`: show sound status and lockout time summary

5 Security-Relevant Behavior

The current implementation already includes several safe-controller protections:

- fixed-length validation for user and admin PINs
- bounded keypad queue capacity
- debounced physical inputs before key acceptance
- rejection of unsupported keys at the queue boundary
- automatic failed-attempt tracking
- timed lockout after the configured threshold
- explicit separation between normal-user and admin workflows

Current prototype limits to be aware of:

- credentials are stored only in volatile memory
- there is no tamper sensing, enclosure switch, or secure element
- PIN comparison is functional rather than hardened against side-channel analysis
- this is a control-panel prototype, not a certified high-security safe product

6 Relevant Source Map

Concern	**Path**
application logic / state machine	`src/app_fsm.c`, `include/app_fsm.h`
hardware boundary	`src/hw.c`, `include/hw.h`
display model	`src/display.c`, `include/display.h`
indicator model	`src/indicators.c`, `include/indicators.h`
keypad queue	`src/keypad.c`, `include/keypad.h`
configuration and limits	`include/config.h`
mutable settings	`src/storage.c`, `include/storage.h`
logic tests	`test/test_safe_logic.c`

7 Summary

This repository implements a compact but well-partitioned embedded safe controller:

- hardware-specific code is isolated in the board support layer
- security and operator behavior are concentrated in a finite-state machine
- logical models decouple policy from display and GPIO details
- the prototype is suitable for demonstration, validation, and later extension toward persistent storage or actuator integration

The Markdown file remains the easiest version-controlled source for review, while this PDF companion is better suited for submission, sharing, and printing.